

Relatório do Trabalho 3 – SISTEMA DE GERENCIAMENTO DE SUPERMERCADO

Universidade Federal do Ceará - Campus de Quixadá

Equipe: Mônica Maria Rodrigues de Castro

Maryana Moraes Sousa

Disciplina: Sistemas Distribuídos

Professor: Rafael Braga

1. Introdução

Este trabalho tem como objetivo o desenvolvimento e a análise de um sistema distribuído baseado no modelo cliente-servidor, aplicado ao contexto de um supermercado. O sistema permite o gerenciamento de funcionários por meio de uma API REST, possibilitando operações como cadastro, listagem, filtragem e cálculo da folha de pagamento. A proposta do trabalho é integrar conceitos de redes, programação e sistemas distribuídos, utilizando diferentes linguagens de programação para os clientes e o servidor.

2. Objetivos

2.1 Objetivo Geral

Desenvolver um sistema cliente-servidor funcional para gerenciamento de funcionários de um supermercado, utilizando comunicação via HTTP e dados no formato JSON.

2.2 Objetivos Específicos

- Implementar uma API REST para gerenciamento de funcionários;
- Desenvolver clientes em C++ e Python para consumir a API;
- Demonstrar o funcionamento do modelo cliente-servidor;
- Aplicar conceitos de sistemas distribuídos e comunicação em rede;
- Realizar testes de conectividade e funcionamento do sistema.

3. Fundamentação Teórica

3.1 Arquitetura Cliente-Servidor

Na arquitetura cliente-servidor, o cliente é responsável por realizar requisições e apresentar os dados ao usuário, enquanto o servidor processa essas requisições e retorna as respostas. Essa arquitetura é amplamente utilizada em sistemas distribuídos por sua escalabilidade e organização.

3.2 API REST

Uma API REST (Representational State Transfer) utiliza o protocolo HTTP para comunicação entre sistemas. As operações são realizadas por meio de métodos como GET e POST, e os dados são geralmente trocados no formato JSON.

3.3 Protocolo HTTP e JSON

O protocolo HTTP define regras de comunicação entre cliente e servidor. O formato JSON (JavaScript Object Notation) é utilizado para representar dados de forma leve e estruturada, facilitando a interoperabilidade entre diferentes linguagens.

4. Tecnologias Utilizadas

- Linguagem C++ (cliente e servidor);
- Linguagem Python (cliente);
- Biblioteca libcurl (cliente C++);
- Biblioteca requests (cliente Python);
- Biblioteca httplib (servidor C++);
- Formato JSON para troca de dados;
- Protocolo HTTP;
- Sistema operacional Linux.

5. Descrição do Sistema

5.1 Servidor

O servidor foi implementado em C++ e é responsável por disponibilizar a API REST. Ele mantém os dados dos funcionários e fornece endpoints para:

- Listagem de funcionários;

- Cálculo da folha de pagamento;
- Listagem dos tipos de funcionários;
- Filtragem por tipo;
- Cadastro de novos funcionários.

```
monica@monica-Aspire-A515-57:~/Área de trabalho/SD/trabalho3/servidor-cpp/build$ ./servidor_http
=====
SERVIDOR HTTP - SUPERMERCADO API
Trabalho 3 - Sistemas Distribuídos
=====
=== INICIALIZANDO SERVIDOR SUPERMERCADO API ===
Adicionando funcionarios de teste...
✓ 4 funcionarios adicionados

✅ SERVIDOR CONFIGURADO COM SUCESSO!

=== INFORMAÇÕES DE ACESSO ===
📍 Local: http://localhost:8080
🌐 Rede WiFi: http://172.25.183.184:8080
🌐 Rede Cabo: http://10.0.109.101:8080
GET /                Página inicial
GET /cadastro        Formulário de cadastro
GET /api/funcionarios Listar todos os funcionários
GET /api/funcionarios/folha Calcular folha de pagamento
GET /api/funcionarios/tipos Listar tipos de funcionários
GET /api/funcionarios/tipo/{tipo} Filtrar por tipo
POST /api/funcionarios Adicionar novo funcionário

=== INSTRUÇÕES PARA ACESSO EM REDE ===
1. Descubra seu IP local: ifconfig ou ipconfig
2. Em outro computador, acesse: http://SEU_IP:8080
3. Para clientes C++/Python, use o IP em vez de localhost

🕒 Aguardando conexões...
```

O servidor processa requisições HTTP, interpreta os dados JSON recebidos e retorna respostas também em JSON.

5.2 Cliente em C++

O cliente em C++ utiliza a biblioteca libcurl para realizar requisições HTTP ao servidor. Ele apresenta um menu interativo no terminal, permitindo ao usuário escolher as operações desejadas. O cliente também realiza testes de conectividade antes de executar as operações.

```
monica@monica-Aspire-A515-57:~/Área de trabalho/SD/traba  
lho3/cliente-cpp/build$ ./cliente
```

```
=====
```

CLIENTE C++ - SUPERMERCADO API

```
=====
```

=== CONFIGURAÇÃO DE REDE ===

Digite o IP do servidor (ou Enter para localhost):

🔍 Testando conexão com http://localhost:8080...

✅ Conexão estabelecida com sucesso!

📖 MENU PRINCIPAL:

1. Listar todos os funcionários
2. Calcular folha de pagamento
3. Listar tipos de funcionários
4. Filtrar por tipo
5. Cadastrar novo funcionário
6. Testar conexão
7. Sair

Escolha uma opção: 5

✚ CADASTRAR NOVO FUNCIONÁRIO

=====

Nome: Rafael

Salário (R\$): 2000.00

Tipo (Vendedor/Gerente/Caixa/Balconista): Gerente

Bônus (R\$): 250.00

📄 JSON enviado: {"nome":"Rafael","salario":2000,"tipo":"Gerente","bonus":250}

✅ Resposta do servidor:

=====

```
{"success": true,"message": "Funcionário cadastrado com sucesso!","id": 5,"nome": "Rafael","tipo": "Gerente","salario": 2000.000000,"total_funcionari  
os": 5,"bonus": 250.000000}
```

=====

Pressione Enter para continuar...

5.3 Cliente em Python

O cliente em Python foi desenvolvido utilizando a biblioteca requests. Assim como o cliente em C++, ele oferece um menu no terminal e permite a comunicação com o servidor por meio de requisições HTTP. O uso do Python facilita a implementação e a leitura do código.

```
monica@monica-Aspire-A515-57:~/Área de trabalho/SD/trabalho3/cliente-python$ python3 cliente.py
```

```
=== CONFIGURAÇÃO DE REDE ===
```

```
Digite o IP do servidor (ou Enter para localhost):
```

```
🔗 Conectando ao servidor: http://localhost:8080
```

```
🔍 Testando conexão com localhost...
```

```
✅ Conexão estabelecida com sucesso!
```

```
=====
```

```
👤 CLIENTE PYTHON - SUPERMERCADO API
```

```
=====
```

1. 📄 Listar todos os funcionários
2. 💰 Calcular folha de pagamento
3. 🏪 Listar tipos de funcionários
4. 🔍 Filtrar por tipo
5. ➕ Cadastrar novo funcionário
6. 🌐 Testar conexão
7. 🚪 Sair

```
=====
```

```
Escolha uma opção: 5
```

```
➕ CADASTRAR NOVO FUNCIONÁRIO
```

```
=====
```

```
Nome: Monica
```

```
Salário (R$): 2000.00
```

```
Tipo (Vendedor/Gerente/Caixa/Balconista): Vendedor
```

```
Comissão (R$): 250.00
```

```
📦 Enviando dados...
```

```
🎉 Funcionário cadastrado com sucesso!
```

```
📄 ID: 6
```

```
👤 Nome: Monica
```

```
📄 Tipo: Vendedor
```

```
⏪ Pressione Enter para continuar...
```

6. Funcionamento do Sistema

O usuário inicia o servidor e, em seguida, executa um dos clientes (C++ ou Python). Após informar o IP do servidor, o cliente testa a conexão. Com a conexão estabelecida, o usuário pode interagir com o sistema por meio do menu, realizando operações como cadastro e consulta de funcionários. Todas as requisições são enviadas ao servidor, que processa e retorna as respostas adequadas.

6.1 Execução do Servidor (C++)

No diretório onde se encontra o código do servidor, o seguinte comando é utilizado para compilar o programa:

```
g++ servidor.cpp -o servidor -std=c++17
```

Após a compilação, o servidor é iniciado com o comando:

```
./servidor
```

O servidor passa a escutar requisições HTTP na porta **8080**.

6.2 Execução do Cliente em C++

No diretório do cliente C++, o código é compilado utilizando:

```
g++ cliente.cpp -o cliente -lcurl -std=c++17
```

Após a compilação, o cliente é executado com:

```
./cliente
```

Durante a execução, o usuário informa o IP do servidor (ou utiliza localhost), e o cliente realiza o teste de conexão antes de exibir o menu interativo.

6.3 Execução do Cliente em Python

Para executar o cliente Python, é necessário possuir o Python 3 instalado, bem como a biblioteca requests . Caso não esteja instalada, pode-se utilizar:

```
pip install requests
```

A execução do cliente é realizada com o comando:

```
python3 cliente.py
```

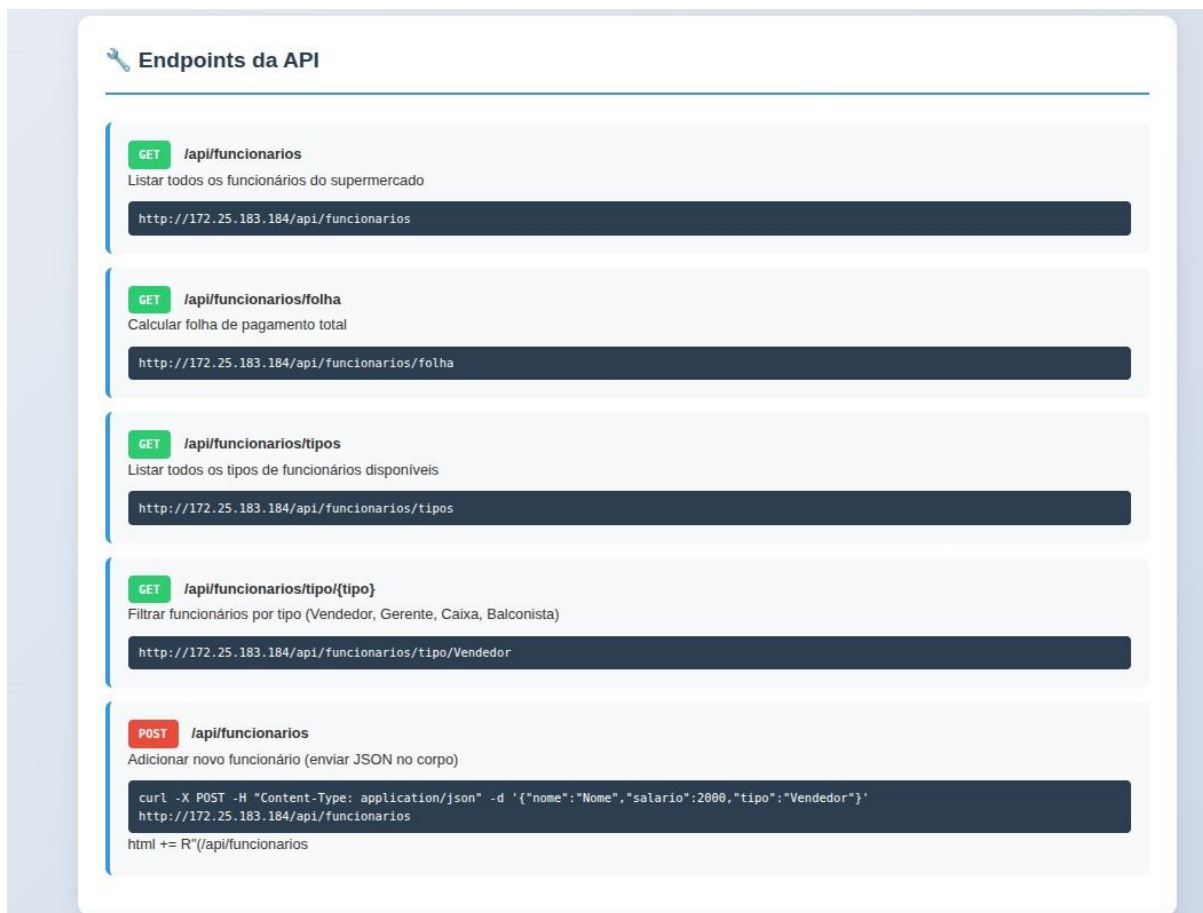
Assim como no cliente C++, o usuário informa o IP do servidor, o sistema testa a conexão e, após isso, permite a interação por meio do menu.

7. Testes Realizados

Foram realizados testes de:

- Conectividade entre cliente e servidor;
- Listagem de funcionários;
- Cadastro de novos funcionários;
- Cálculo da folha de pagamento;
- Filtragem por tipo de funcionário.





Os testes demonstraram que o sistema funciona corretamente quando cliente e servidor estão na mesma rede e o servidor está em execução.

8. Resultados e Discussão

O sistema apresentou funcionamento adequado, permitindo a comunicação entre diferentes clientes e o servidor. A utilização de linguagens distintas evidenciou a interoperabilidade proporcionada pelo uso de APIs REST e JSON. O projeto reforçou conceitos importantes de sistemas distribuídos e programação em rede.

9. Conclusão

Com a realização deste trabalho, foi possível compreender na prática o funcionamento da arquitetura cliente-servidor e a implementação de uma API REST. O sistema desenvolvido atendeu aos objetivos propostos, demonstrando a comunicação eficiente entre clientes e servidor, além da aplicação de conceitos fundamentais de sistemas distribuídos.

